

Robótica e Automação - Trabalho Nº2

Leonardo Soares da Costa Tanaka - DRE: 121067652

Engenharia de Controle e Automação/UFRJ

Rio de Janeiro, Brasil

Dezembro de 2024

1 Introdução

Para a resolução do trabalho, foi o manipulador Kinova Gen3 Ultra lightweight (www.kinovarobotics.com) da Kinova Inc.. A estrutura cinemática deste manipulador de 7 juntas pode ser vista na figura:

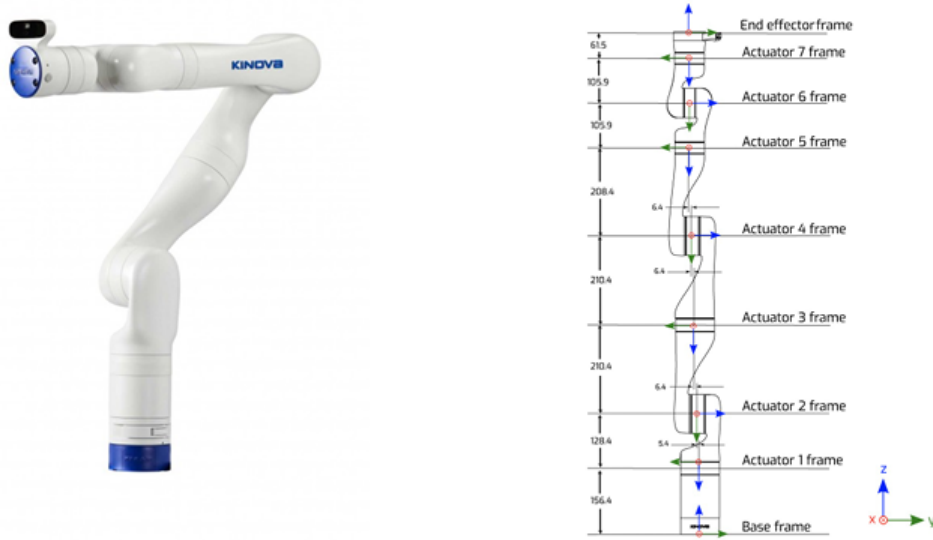


Figura 1: Foto e Diagrama do manipulador Kinova

Além disso, os parâmetros de Denavit-Hartenberg do manipulador são apresentados na Tabela 1. Foi considerado que a orientação do sistema de coordenadas 0 com relação à base é dado por $R_{b0} = R_x(\pi)$ onde $R_x(\cdot)$ é uma rotação elementar ao redor do eixo x.

Para a primeira parte do trabalho, foi considerado a posição inicial do manipulador é definida pelos ângulo das juntas: $\theta = [0 \ 15 \ 180 \ -130 \ 0 \ 55 \ 90]$ (graus). Porém, para a segunda parte do trabalho, foi considerado a posição inicial do manipulador é definida pelos ângulos das juntas: $\theta_h = [90 \ 15 \ 180 \ -130 \ 0 \ 55 \ 0]$ (graus). Além disso, foi considerado que na tabela de Denavit-Hartenberg da primeira parte do trabalho tem o $d_7 = -0.3574m$.

Junta	α (rad)	A (m)	θ (rad)	D (m)	Offset (rad)
1	$\pi/2$	0	θ_1	-0.2848	π
2	$\pi/2$	0	θ_2	-0.0118	π
3	$\pi/2$	0	θ_3	-0.4208	π
4	$\pi/2$	0	θ_4	-0.0128	π
5	$\pi/2$	0	θ_5	-0.3143	π
6	$\pi/2$	0	θ_6	0	π
7	π	0	θ_7	0	π

Tabela 1: Parâmetros de Denavit-Hartenberg Standard do Kinova Gen3

2 Desenvolvimento

2.1 Controle Cinemático de Manipuladores

Foi considerado o controle cinemático da posição do punho do manipulador em que somente a Sinais de controle é controlada, porque as 3 últimas juntas estão na mesma posição fazendo que as três últimas colunas do jacobiano sejam zero. Além disso, a velocidade máxima das juntas foi definida como $\dot{\theta}_{max} = 1 \text{ rad/s}$. Foram consideradas as seguintes trajetórias (com $\omega_n = \pi/2$):

(a) $\mathbf{x}_d$ descreve uma circunferência de 0.1 m de raio no plano $X - Z$, centrada no ponto $\mathbf{x}_0 = \begin{bmatrix} 0.4 & 0 & 0.4 \end{bmatrix}^T$, e percorrida com uma frequência ω_n .

(b) A circunferência do item anterior projetada no plano definido pela equação: $(x - 0.4) - y + (z - 0.4) = 0$.

(c) Considerando agora $\omega_n = \frac{\pi}{8}$, temos: $\mathbf{x}_d(t) = \begin{bmatrix} 0.08(\sin(\omega_n t) + \sin(4\omega_n t)) + 0.4 \\ 0 \\ 0.08(\cos(\omega_n t) + \cos(4\omega_n t)) + 0.4 \end{bmatrix}$.

Foi simulado o comportamento do sistema em malha fechada para as trajetórias especificadas utilizando os seguinte controladores:

1. $\mathbf{u} = (J(\boldsymbol{\theta}))^\dagger \mathbf{K}(\mathbf{x}_d - \mathbf{x})$.
2. $\mathbf{u} = (J(\boldsymbol{\theta}))^\dagger [\dot{\mathbf{x}}_d + \mathbf{K}(\mathbf{x}_d - \mathbf{x})]$, onde $J^\dagger = J^\top (JJ^\top)^{-1}$.

Para implementar o segundo controlador, foi feita a derivação algébrica das três trajetórias consideradas no problema. Portanto, não foi utilizado derivação numérica para definir o $\dot{x}_d$ no segundo controlador. Além disso, foi sintonizado o ganho $\mathbf{K}$ com 6 configurações diferentes para obter o melhor desempenho possível atendendo a especificação de $|u_i| \leq 1 (i = 1, 2, 3, 4)$. A seguir temos os gráficos utilizados para analisar as 6 configurações diferentes:

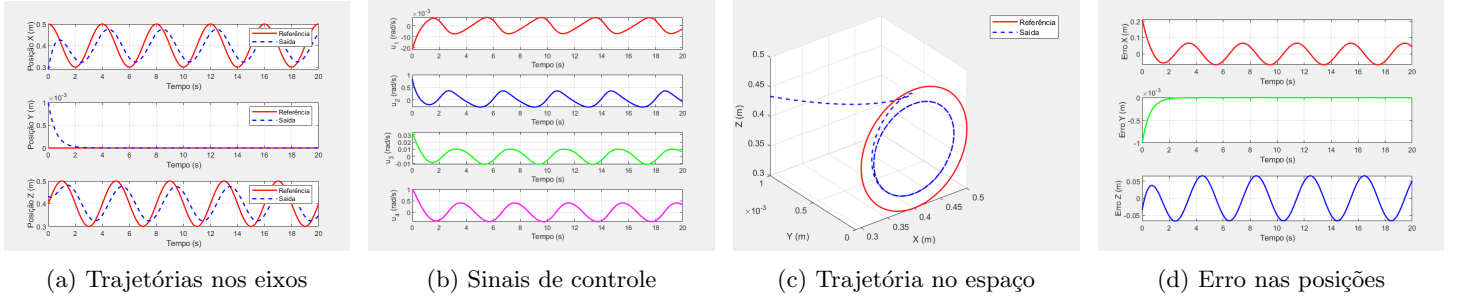


Figura 2: Gráficos da Trajetória (a) com Controlador 1 ($K = 1.81$)

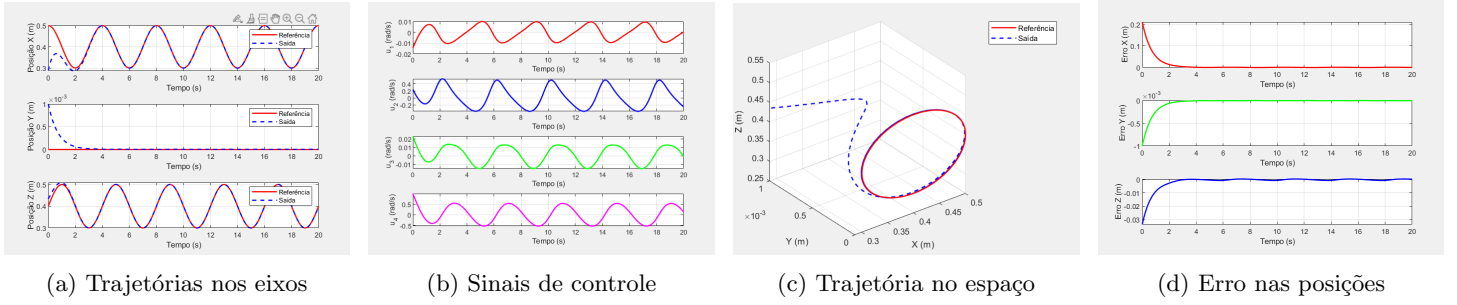


Figura 3: Gráficos da Trajetória (a) com Controlador 2 ($K = 1.39$)

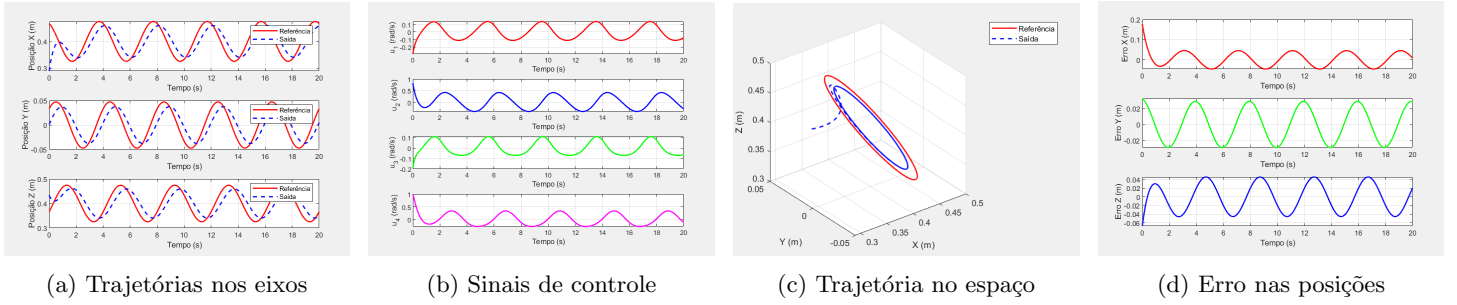


Figura 4: Gráficos da Trajetória (b) com Controlador 1 ($K = 2.05$)

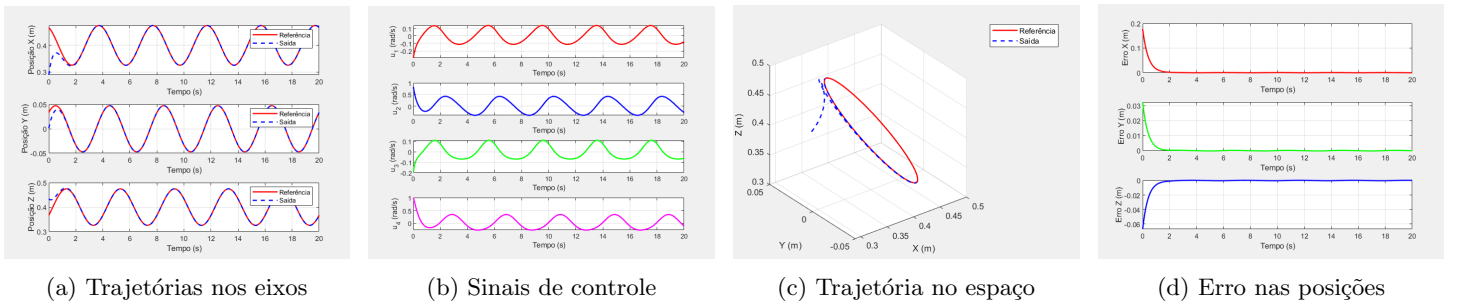


Figura 5: Gráficos da Trajetória (b) com Controlador 2 ($K = 2.42$)

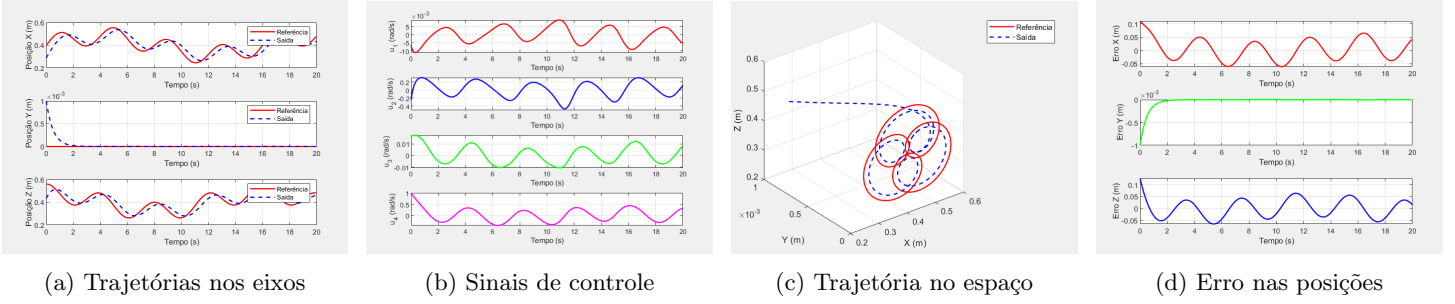


Figura 6: Gráficos da Trajetória (c) com Controlador 1 ($K = 1.99$)

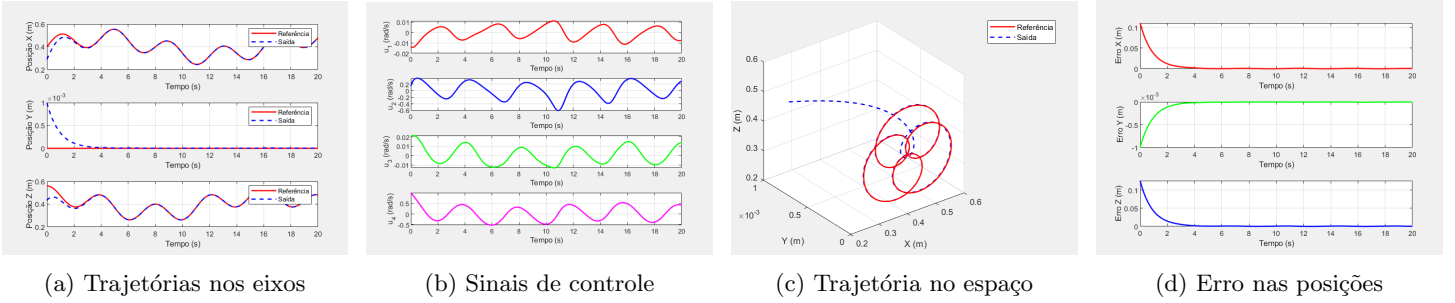


Figura 7: Gráficos da Trajetória (c) com Controlador 2 ($K = 2.17$)

Após analisar os dados obtidos das 6 configurações diferentes, é possível concluir sobre a estabilidade e desempenho de cada um dos controladores. Portanto, os controladores podem assegurar a estabilidade, se $J(\theta)$ tenha posto completo, garantindo a existência do pseudo-inverso ($J^\dagger$). Além disso, o controlador 1 não consegue ter a convergência ao ser respeitado as especificações do problema. Assim, a saída acaba tendo uma diferença de fase em relação a trajetória de referência e um desvio considerável para as três trajetórias especificadas, o que demonstra um desempenho ruim para seguimento de trajetórias. Entretanto, o controlador 2 consegue ter a convergência ao ser respeitado as especificações do problema. Ademais, o erro entre a saída e a trajetória de referência acaba tendendo a zero de uma forma bem rápida, tendo um desempenho bom para seguimento de trajetórias. Isso acontece, porque o controlador 2 possui uma feedforward ($\dot{x}_d$) compensando o movimento desejado.

2.2 Controle Cinemático de Manipuladores por Servo Visão

Para o controle cinemático da posição do efetuador utilizando servo-visão, foi considerado somente os comandos de velocidade das juntas 1,2,4,6. Assim, considerando as juntas 3,5,7 fixas, i.e. $\dot{\theta}_3 = \dot{\theta}_5 = \dot{\theta}_7 = 0$. Além disso, foi utilizada uma câmera monocular com eixo ótico (x) é perpendicular ao plano de trabalho (x-y) do manipulador, a distância focal é $f = 4mm$, posição a uma distância de $z_0 = 0.4338m$ do plano de trabalho do manipulador e o fator de escala é $\alpha = 200000$ pixels/m nas duas direções. Ademais, foi definido a origem do plano da imagem $O_c = \begin{bmatrix} 320 & 240 \end{bmatrix}^T$ e a posição da câmera com relação à base do manipulador é $P_{bc} = \begin{bmatrix} 0 & -0.603 & 0 \end{bmatrix}^T$ com orientação $\phi = 0$, i.e. $R_{bc} = I$. Foi considerada seguinte trajetória em coordenada da tela (com $\omega_n = 0.5$):

$$x_{cd} = \begin{bmatrix} 360 - 30 \cos(\omega_n t) - 30 \cos(1.5\omega_n t) - 10 \cos(2\omega_n t) \\ 150 - 20 \cos(\omega_n t + 1.6) - 20 \cos(1.5\omega_n t + 1.6) - 10 \cos(2\omega_n t + 1.6) \end{bmatrix} \quad (1)$$

Também, foi feito a derivação algébrica da trajetória para utilizar o controle 2 da subseção anterior:

$$\dot{x}_{cd} = \begin{bmatrix} 30 \sin(\omega_n t)\omega_n + 45 \sin(1.5\omega_n t)1.5\omega_n + 20 \sin(2\omega_n t)2\omega_n \\ 20 \sin(\omega_n t + 1.6)\omega_n + 30 \sin(1.5\omega_n t + 1.6)1.5\omega_n + 20 \sin(2\omega_n t + 1.6)2\omega_n \end{bmatrix} \quad (2)$$

A partir das informações acima foi feita um estudo em relação ao controle por servo-visão. Assim, temos a seguir certos conceitos que foram necessários para fazer o código de solução do problema no Matlab. A transformação câmera/espaco de trabalho é dada por:

$$\mathbf{x}_c = K_p \cdot \mathbf{x} + \mathbf{x}_{c0} \quad (3)$$

onde

$$K_p = \frac{f}{z_0} \begin{bmatrix} \alpha_1 & 0 \\ 0 & \alpha_2 \end{bmatrix} \quad (4)$$

e

$$\mathbf{x}_{c0} = \frac{f}{z_0} \begin{bmatrix} \alpha_1 & 0 \\ 0 & \alpha_2 \end{bmatrix} \begin{bmatrix} \cos(\phi) & \sin(\phi) \\ \sin(\phi) & -\cos(\phi) \end{bmatrix} P_{bc} + \begin{bmatrix} O_1 \\ O_2 \end{bmatrix} \quad (5)$$

em que

- $\mathbf{x}_c$: posição do efetuador no sistema de coordenadas da imagem.
- $\mathbf{x}_c^0$: offset da posição do frame da câmera com respeito ao do robô.
- K_p : orientação e escalamento da câmera com respeito ao robô.
- ϕ : ângulo de orientação da câmera com respeito ao frame do robô.
- f : distância focal da câmera.
- z_0 : profundidade do plano da imagem ao plano de trabalho do robô.
- $\alpha_i > 0$: fatores de escala (pixels/m).

No sistema de coordenadas da câmera, a cinemática diferencial é dada por:

$$\dot{\mathbf{x}}_c = K_p \dot{\mathbf{x}}, \quad \mathbf{x}^T = [x_1, x_2] \quad (6)$$

em que $\mathbf{x}_{cd} \in \mathbb{R}^2$ é a trajetória desejada do efetuador no plano da câmera. Uma lei de controle cartesiano pode ser projetada para que $\mathbf{x}_c \rightarrow \mathbf{x}_{cd}$, ou seja, $e_{cd} \rightarrow 0$. A equação dinâmica de e_{cd} é dada por:

$$\dot{e}_{cd} = \dot{x}_{cd} - \dot{\mathbf{x}}_c = \dot{x}_{cd} - K_p J(\theta) u \quad (7)$$

Com uma lei de controle dada por:

$$u = (K_p J(\theta))^{-1} [\dot{x}_{cd} + K(\mathbf{x}_{cd} - \mathbf{x}_c)] \quad (8)$$

O sistema em malha fechada é dado por:

$$\dot{e}_{cd} + K e_{cd} = 0 \quad \text{para } K > 0 \quad (9)$$

O sistema é exponencialmente estável, com a solução:

$$e_{cd}(t) = e^{-Kt} e_{cd}(0) \quad (10)$$

Primeiramente, foi simulado o controle por servo-visão do efetuador com $K = 1$ para avaliar a estabilidade, tendo os seguintes resultados:

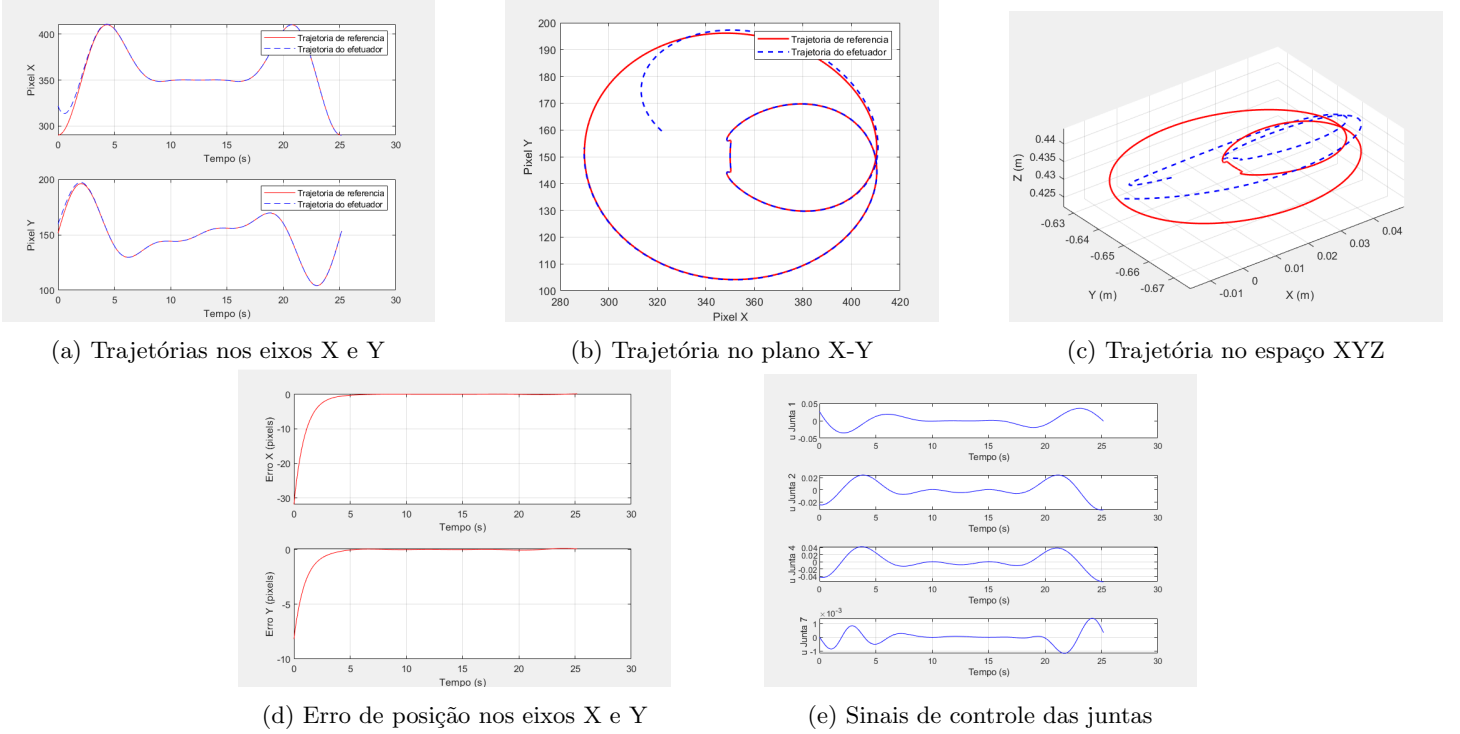
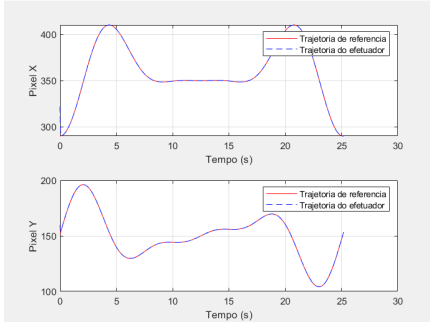
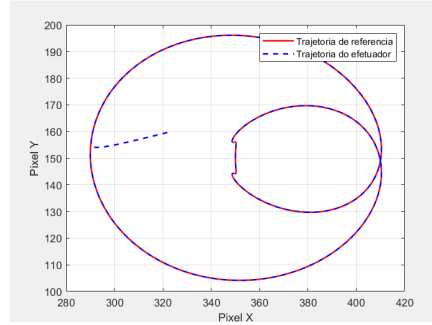


Figura 8: Gráficos do controle por servo-visão ($K = 1$)

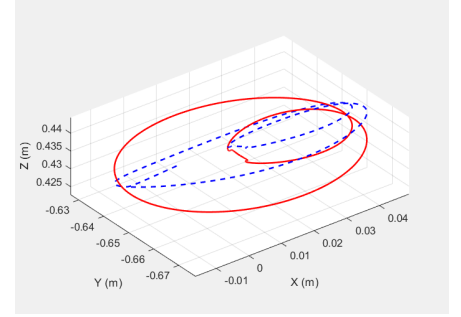
É possível notar que o efetuador consegue seguir a trajetória de referência. Além disso, o gráfico de erro consegue mostrar que o sistema é exponencialmente estável. Entretanto, para o ganho do controlador sendo $K = 1$, ainda tem um grande margem de aumento desse ganho para ficar mais próximos dos limites de $|u_i| \leq 1 (i = 1, 2, 4, 7)$. Então, foi feito a sintonização do ganho do controlador satisfazendo a expressão anterior, obtendo $K = 37.25$ e tendo os seguintes gráficos:



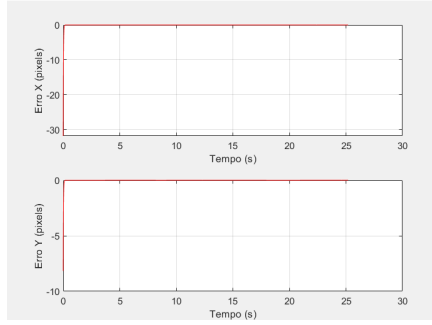
(a) Trajetórias nos eixos X e Y



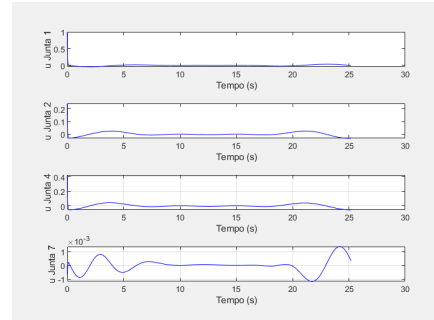
(b) Trajetória no plano X-Y



(c) Trajetória no espaço XYZ



(d) Erro de posição nos eixos X e Y



(e) Sinais de controle das juntas

Figura 9: Gráficos do controle por servo-visão ($K = 37.25$)

É possível notar que o erro vai muito mais rápido para zero, mas possui um sinal de controle das juntas muito maior no começo do seguimento da trajetória.

3 Conclusão

Apêndice

Listing 1: Código da primeira parte do trabalho

```

1 clear all;
2
3 % Definição dos parâmetros DH com juntas revolutas
4 L(1) = Revolute('d', -0.2848, 'a', 0, 'alpha', pi/2, 'offset', 0);
5 L(2) = Revolute('d', -0.0118, 'a', 0, 'alpha', pi/2, 'offset', pi);
6 L(3) = Revolute('d', -0.4208, 'a', 0, 'alpha', pi/2, 'offset', pi);
7 L(4) = Revolute('d', -0.0128, 'a', 0, 'alpha', pi/2, 'offset', pi);
8 L(5) = Revolute('d', -0.3143, 'a', 0, 'alpha', pi/2, 'offset', pi);
9 L(6) = Revolute('d', 0, 'a', 0, 'alpha', pi/2, 'offset', pi);
10 L(7) = Revolute('d', 0, 'a', 0, 'alpha', pi, 'offset', pi);
11
12 % Criação do manipulador
13 kinova = SerialLink(L, 'name', 'Kinova');
14 kinova.base = trotx(180); % Define a base do robô
15
16 % Condições iniciais
17 q0 = deg2rad([0, 15, 180, -130, 0, 55, 90]);
18
19 % Inicialização de variáveis
20 qi = q0;
21 h = 0.01; % Passo de tempo
22 tmax = 10; % Tempo máximo (s)
23 wmax = 1; % Velocidade máxima das juntas (rad/s)
24 wn = pi/2; % Frequência natural
25 wn3 = pi/8; % Frequência natural - Terceira Trajetoria
26
27 K = 1.81; % Ganho do controlador 1 da trajetória (a)
28 % K = 2.05; % Ganho do controlador 1 da trajetória (b)
29 % K = 1.99; % Ganho do controlador 1 da trajetória (c)
30 % K = 1.39; % Ganho do controlador 2 da trajetória (a)
31 % K = 2.42; % Ganho do controlador 2 da trajetória (b)
32 % K = 1.10; % Ganho do controlador 2 da trajetória (c)
33
34 % Vetores para armazenamento dos dados
35 t_vec = 0:h:tmax;
36 x_ref = zeros(3, length(t_vec));
37 x_out = zeros(3, length(t_vec));
38 u_vec = zeros(4, length(t_vec)); % Para armazenar os 4 primeiros valores de u
39
40 % Simulação
41 for k = 1:length(t_vec)
42     t = t_vec(k);

```



```

43
44 % Jacobiano e cinemática direta
45 J = kinova.jacob0(qi); % Jacobiano do robô
46 Jp = J(1:3,1:7);
47 x = kinova.fkine(qi).t; % Posição atual do fim do braço
48
49 % Selecionar a trajetória desejada:
50 xdd=0;
51
52 % Trajetória (a)
53 xd = routeGen1(t, wn);
54 % xdd = routeGen1Deriv(t, wn);
55
56 % Trajetória (b)
57 % xd = routeGen2(t, wn);
58 % xdd = routeGen2Deriv(t, wn);
59
60 % Trajetória (c)
61 % xd = routeGen3(t, wn3);
62 % xdd = routeGen3Deriv(t, wn3);
63
64 % Erro de posição e velocidade desejada
65 err = xd - x; % Erro de posição
66
67 % Controlador
68 u = pinv(Jp) * (xdd + K * err); % Cálculo das velocidades para as primeiras 4 juntas
69
70 % Verificar se algum valor de u é maior que 1
71 if any(abs(u) > 1)
72     disp('Alerta: Um ou mais valores de u excedem 1.');
```

73 disp('Valores de u:');

74 disp(u);

75 end

76

77 % Integração (Euler) para as primeiras 4 juntas

78 qk = qi; % Copia o vetor atual de juntas

79 qk = qi + h * u'; % Atualiza apenas as primeiras 4 juntas

80 qi = qk; % Atualiza o estado

81

82 % Armazenar dados

83 x_ref(:, k) = xd; % Trajetória de referência

84 x_out(:, k) = x; % Trajetória saída

85 u_vec(:, k) = u(1:4); % Armazena os 4 primeiros valores de u

86 end

87

88 % Plotagem dos resultados

```

89 figure;
90 subplot(3, 1, 1);
91 plot(t_vec, x_ref(1, :), 'r', 'LineWidth', 1.5); hold on;
92 plot(t_vec, x_out(1, :), 'b--', 'LineWidth', 1.5);
93 xlabel('Tempo (s)');
94 ylabel('Posição X (m)');
95 legend('Referência', 'Saída');
96 title('Trajetoria no eixo X');
97 grid on;
98
99 subplot(3, 1, 2);
100 plot(t_vec, x_ref(2, :), 'r', 'LineWidth', 1.5); hold on;
101 plot(t_vec, x_out(2, :), 'b--', 'LineWidth', 1.5);
102 xlabel('Tempo (s)');
103 ylabel('Posição Y (m)');
104 legend('Referência', 'Saída');
105 title('Trajetoria no eixo Y');
106 grid on;
107
108 subplot(3, 1, 3);
109 plot(t_vec, x_ref(3, :), 'r', 'LineWidth', 1.5); hold on;
110 plot(t_vec, x_out(3, :), 'b--', 'LineWidth', 1.5);
111 xlabel('Tempo (s)');
112 ylabel('Posição Z (m)');
113 legend('Referência', 'Saída');
114 title('Trajetoria no eixo Z');
115 grid on;
116
117 % Gráfico 3D da trajetória real e desejada
118 figure;
119 plot3(x_ref(1, :), x_ref(2, :), x_ref(3, :), 'r-', 'LineWidth', 1.5); hold on;
120 plot3(x_out(1, :), x_out(2, :), x_out(3, :), 'b--', 'LineWidth', 1.5);
121 xlabel('X (m)');
122 ylabel('Y (m)');
123 zlabel('Z (m)');
124 legend('Referência', 'Saída');
125 grid on;
126
127 % Gráficos das Sinais de controle
128 figure;
129
130 subplot(4, 1, 1);
131 plot(t_vec, u_vec(1, :), 'r', 'LineWidth', 1.5);
132 xlabel('Tempo (s)');
133 ylabel('u_1 (rad/s)');
134 title('Velocidade da Junta 1');

```

```

135 grid on;
136
137 subplot(4, 1, 2);
138 plot(t_vec, u_vec(2, :), 'b', 'LineWidth', 1.5);
139 xlabel('Tempo (s)');
140 ylabel('u_2 (rad/s)');
141 title('Velocidade da Junta 2');
142 grid on;
143
144 subplot(4, 1, 3);
145 plot(t_vec, u_vec(3, :), 'g', 'LineWidth', 1.5);
146 xlabel('Tempo (s)');
147 ylabel('u_3 (rad/s)');
148 title('Velocidade da Junta 3');
149 grid on;
150
151 subplot(4, 1, 4);
152 plot(t_vec, u_vec(4, :), 'm', 'LineWidth', 1.5);
153 xlabel('Tempo (s)');
154 ylabel('u_4 (rad/s)');
155 title('Velocidade da Junta 4');
156 grid on;
157
158 % Funções de geração de trajetória
159 function xd = routeGen1(t, wn)
160     % Trajetória (a): Circunferência no plano X-Z
161     r = 0.1; % Raio da circunferência
162     x0 = [0.4; 0; 0.4]; % Centro da circunferência
163     xd = x0 + [r * cos(wn * t); 0; r * sin(wn * t)];
164 end
165
166 function xdot = routeGen1Deriv(t, wn)
167     % Derivada da trajetória (a): Velocidade da circunferência no plano X-Z
168     r = 0.1; % Raio da circunferência
169     xdot = [-r * wn * sin(wn * t); 0; r * wn * cos(wn * t)];
170 end
171
172 function xd = routeGen2(t, wn)
173     % Trajetória (b): projeção de (a) no plano (x - 0.4) - y + (z - 0.4) = 0
174     r = 0.1; % Raio da circunferência
175     x0 = [0.4; 0; 0.4]; % Centro da circunferência
176     n = [1; -1; 1] * 1 / sqrt(3); % Normal do plano
177     P = eye(3) - n * n'; % Operador de projeção no plano
178     xd = x0 + P * [r * cos(wn * t); 0; r * sin(wn * t)]; % Trajetória projetada no plano
179 end
180

```

```
181 function xdot = routeGen2Deriv(t, wn)
182     % Derivada da trajetoria (b): Velocidade projetada no plano inclinado
183     r = 0.1; % Raio da circunferência
184     n = [1; -1; 1] * 1 / sqrt(3); % Normal do plano
185     P = eye(3) - n * n'; % Operador de projeção no plano
186
187     % Derivada dos componentes
188     vel = [-r * wn * sin(wn * t); 0; r * wn * cos(wn * t)];
189     xdot = P * vel; % Projeção da velocidade no plano
190 end
191
192 function xd = routeGen3(t, wn)
193     % Trajetoria (c): Oscilação complexa no plano X-Z
194     xd = [0.08 * (sin(wn * t) + sin(4 * wn * t)) + 0.4;
195          0;
196          0.08 * (cos(wn * t) + cos(4 * wn * t)) + 0.4];
197 end
198
199 function xdot = routeGen3Deriv(t, wn)
200     % Derivada da trajetoria (c): Velocidade da oscilação no plano X-Z
201     xdot = [0.08 * (wn * cos(wn * t) + 4 * wn * cos(4 * wn * t));
202            0;
203            -0.08 * (wn * sin(wn * t) + 4 * wn * sin(4 * wn * t))];
204 end
```